

SatsPath × Arkade PWA — Technical Integration Plan (MVP)

Goal: Arkade PWA sends `alice@domain.com + amount_sats` → receives `QuoteResponse` with payment payload + QR → executes payment via LDK (Lightning) / Arkade SDK (Ark) / BDK (on-chain).

Architecture: PWA (TypeScript) handles resolution → `satspath-wasm` verifies profile signature → Router (TypeScript port) selects rail → Arkade SDK validates VTXO on receive.

1. Current State Inventory

SatsPath (Rust)

Crate	Exports	WASM Ready?
<code>satspath-core</code>	<code>PaymentMethod</code> , <code>PaymentProfile</code> , <code>SignedPaymentProfile</code> , <code>verify_signed_profile</code> , resolvers (BIP353, HTTP, Nostr), router types	Uses <code>tokio</code> , <code>reqwest</code> , <code>secp256k1</code> (no WASM)
<code>satspath-router</code>	<code>quote()</code> , <code>build_qr_payload()</code> , <code>select_route()</code> , <code>ArkRoutePlan</code> , <code>FeeEstimate</code>	Same deps
<code>satspath-wasm</code>	<code>verify_signed_profile</code> , <code>canonical_profile_json</code> , <code>topic_for_alias</code>	Only crypto primitives
<code>satspathd</code>	REST API <code>/v1/quote</code> , <code>/v1/pay</code> , <code>/v1/resolve</code> , <code>/v1/dns/resolve</code>	N/A (daemon)

Arkade SDK (TypeScript)

Module	Purpose
<code>vtxoDAGVerification.ts</code>	Tier 1: DAG reconstruction, signature + taproot + timelock + hash preimage validation
<code>sovereignStorage.ts</code>	Local encrypted storage for sovereign exit data
<code>arkdProvider.ts</code>	Indexer + on-chain provider interfaces

2. MVP Scope (4–6 weeks)

In Scope

- PWA resolves `alias@domain` via **TypeScript resolvers** (BIP-353 DoH, HTTPS well-known, Nostr NIP-05)
- `satspath-wasm` verifies `SignedPaymentProfile` signature (`secp256k1 Schnorr`)
- **Router ported to TypeScript** (~300 LOC): fee fetch → scoring → rail selection → payment payload
- `QuoteResponse` returned to UI (`snake_case` JSON, matches Rust exactly)
- Ark receive: Arkade SDK validates VTXO DAG + sovereign exit ready
- Lightning send: PWA calls LDK/Breez SDK with BOLT11 invoice
- On-chain send: PWA calls BDK/rust-bitcoin via WASM or builds BIP-21 URI

Out of Scope (v2)

- Full WASM port of resolver chain + router
 - UniFFI bindings for native mobile
 - BOLT12, Silent Payments, Split Payments
 - Key rotation / profile revocation
 - Email/platform verification (invite flow only)
-

3. Detailed Work Breakdown

3.1 TypeScript Resolvers (New Package: @satspath/resolvers)

Location: New npm package or satspath/sdk/resolvers/

Interfaces (mirror Rust ProfileResolver trait):

```
// packages/resolvers/src/types.ts
export interface SignedPaymentProfile {
  profile: PaymentProfile;
  signature: string; // hex schnorr sig
}

export interface PaymentProfile {
  alias: string;
  identity_pubkey: string; // hex compressed
  methods: PaymentMethod[];
  updated_at: number;
  expires_at?: number;
  sequence?: number;
  preferences: string[];
  nonce?: string;
  rotation?: KeyRotation;
  method_verifications: MethodVerification[];
}

export type PaymentMethod =
  | { type: "Onchain"; label: string; network: "mainnet" | "testnet" | "regtest"; address?: string; sil
  | { type: "Lightning"; label: string; lightning_address?: string; lnurl?: string; bolt12?: string; re
  | { type: "Ark"; label: string; server: string; pubkey: string; vtxo_pointer?: string; opaque_uri?: s

export interface ProfileResolver {
  resolve_alias(alias: string): Promise<SignedPaymentProfile>;
}
```

Resolvers to implement (in order, first success wins): 1. **LocalRegistryResolver** — reads localStorage["satspath:registry"] (JSON array of SignedPaymentProfile) 2. **Bip353Resolver** — DNS-over-HTTPS (Google/Cloudflare) for _bitcoin.payment.<domain> TXT → parse BIP-321 URI 3. **HttpWellKnownResolver** — GET https://<domain>/.well-known/satspath/<alias>.json → SignedPaymentProfile 4. **NostrNip05Resolver** — GET https://<domain>/.well-known/nostr.json?name=<user> → { names: { <user>: <pubkey> }, relays: { <pubkey>: [relay...] } } - Query relays (NIP-01 REQ) for kind 30078 with d = "satspath-profile:<alias>" - Verify event author pubkey matches NIP-05 pubkey 5. **P2PResolver** (optional) — call local satspatd via HTTP if running (tauri sidecar)

All resolvers: return SignedPaymentProfile or throw AliasNotFound.

3.2 satspath-wasm Extensions (Crypto Only)

File: satspath/crates/satpath-wasm/src/crypto.rs

Add exports:

```
#[wasm_bindgen]
pub fn verify_signed_profile(profile_json: &str) -> Result<bool, JsValue> {
    let signed: SignedPaymentProfile = serde_json::from_str(profile_json)?;
    Ok(verify_signed_profile(&signed)?)
}

#[wasm_bindgen]
pub fn canonical_profile_json(profile_json: &str) -> Result<String, JsValue> {
    let profile: PaymentProfile = serde_json::from_str(profile_json)?;
    Ok(canonical_json::to_string(&profile)?)
}

#[wasm_bindgen]
pub fn fingerprint_pubkey(pubkey_hex: &str) -> Result<String, JsValue> {
    let pk = hex::decode(pubkey_hex)?;
    Ok(fingerprint_pubkey(&pk)?)
}
```

Build: `wasm-pack build --target web --out-dir ../../sdk/wasm/pkg`

NPM: Publish @satspath/wasm or copy pkg/ to PWA src/wasm/

3.3 Router Port to TypeScript (New Package: @satspath/router)

Location: satspath/sdk/router/ or new npm package

Core Logic (port from router.rs, ark_routes.rs, scoring.rs, priority.rs):

```
// packages/router/src/types.ts (mirrors Rust exactly)
export interface RouteRequest {
    alias: string;
    amount_sats: number;
    signed_profile: SignedPaymentProfile;
    urgency: "low" | "normal" | "high";
    max_fee_sats?: number;
    max_fee_percent?: number;
}

export interface FeeEstimate {
    fastest_fee: number; // sat/vB
    half_hour_fee: number;
    hour_fee: number;
    economy_fee: number;
    minimum_fee: number;
}

export interface RouteQuote {
    selected_method: PaymentMethod;
    estimated_fee_sats: number;
    estimated_confirmation: string; // "instant" | "~10 min" | "~60 min" | "ark_vtxo"
```

```

    reason: string;
    execution: ExecutionMode;
    wallet_hint: string;
}

export type ExecutionMode =
  | { type: "Preview" }
  | { type: "MainnetPreview" }
  | { type: "TestnetExperimental" }
  | { type: "ManualWallet" };

export type SwapDirective =
  | { type: "LightningPayment"; target_ln_address?: string }
  | { type: "SubmarineSwap"; target_invoice?: string }
  | { type: "ReverseSwap"; target_address?: string; silent_payment_pubkey?: string }
  | { type: "ChainSwap"; target_address?: string; silent_payment_pubkey?: string }
  | { type: "ArkTransfer"; server: string; pubkey: string }
  | { type: "ArkadeManual" };

```

Routing Algorithm (exact port of select_route_with_fees):

```

// packages/router/src/router.ts
const LIGHTNING_THRESHOLD_SATS = 100_000;

export async function selectRoute(req: RouteRequest, fees: FeeEstimate): Promise<RouteQuote> {
  const { signed_profile, amount_sats } = req;
  const methods = signed_profile.profile.methods;

  // 1. Lightning for small amounts if available
  if (amount_sats < LIGHTNING_THRESHOLD_SATS) {
    const ln = methods.find(m => m.type === "Lightning");
    if (ln) return lightningQuote(ln, amount_sats);
  }

  // 2. On-chain if fees acceptable (hour_fee * 1.1 <= 10 sat/vB)
  const onchain = methods.find(m => m.type === "Onchain");
  if (onchain) {
    const effectiveFee = Math.ceil(fees.hour_fee * 1.1);
    if (effectiveFee <= 10) {
      return onchainQuote(onchain, amount_sats, fees);
    }
    // High fees but no alternative + still onchain with warning
    if (!methods.some(m => m.type === "Ark" || m.type === "Lightning")) {
      return onchainQuote(onchain, amount_sats, fees);
    }
  }

  // 3. Ark fallback (testnet only)
  const ark = methods.find(m => m.type === "Ark");
  if (ark) {
    return arkQuote(ark, amount_sats);
  }

  // 4. On-chain anyway (last resort)
  if (onchain) return onchainQuote(onchain, amount_sats, fees);
}

```

```

    throw new Error("NoRouteFound: no usable payment method");
}

function lightningQuote(method: PaymentMethod, amount: number): RouteQuote {
    return {
        selected_method: method,
        estimated_fee_sats: Math.max(1, Math.floor(amount * 0.0001)), // ~10 ppm
        estimated_confirmation: "instant",
        reason: `Amount (${amount} sats) below ${LIGHTNING_THRESHOLD_SATS} threshold and Lightning available`,
        execution: { type: "ManualWallet" }, // PWA calls LDK/Breez
        wallet_hint: "Use LDK, Breez SDK, or any LN wallet to pay the invoice."
    };
}

function onchainQuote(method: PaymentMethod, amount: number, fees: FeeEstimate): RouteQuote {
    const feeRate = Math.ceil(fees.hour_fee * 1.1);
    const feeSats = estimateOnchainFee(method, amount, feeRate);
    return {
        selected_method: method,
        estimated_fee_sats: feeSats,
        estimated_confirmation: "~60 min",
        reason: `On-chain fee ${feeRate} sat/vB (hour fee ${fees.hour_fee} * 1.1).`,
        execution: { type: "ManualWallet" },
        wallet_hint: "Use any BIP-21 compatible wallet (BlueWallet, Sparrow, Electrum, etc.)"
    };
}

function arkQuote(method: PaymentMethod, amount: number): RouteQuote {
    return {
        selected_method: method,
        estimated_fee_sats: 0, // Ark server absorbs
        estimated_confirmation: "ark_vtxo",
        reason: "High on-chain fees; Ark selected as fallback (testnet preview).",
        execution: { type: "TestnetExperimental" }, // Arkade SDK handles
        wallet_hint: "Arkade SDK will execute VTXO transfer. Testnet only."
    };
}

function estimateOnchainFee(method: PaymentMethod, amount: number, feeRate: number): number {
    // ~140 vbytes for 1-in-2-out taproot
    const vsize = 140;
    return Math.ceil(vsize * feeRate);
}

```

Fee Fetching (port fees.rs):

```

// packages/router/src/fees.ts
export async function fetchFeeEstimate(): Promise<FeeEstimate> {
    const res = await fetch("https://mempool.space/api/v1/fees/recommended");
    if (!res.ok) throw new Error("Fee fetch failed");
    return res.json(); // matches FeeEstimate shape
}

```

3.4 QuoteResponse Builder (TypeScript)

File: packages/router/src/quote.ts

```
import { selectRoute, fetchFeeEstimate } from "../router";
import { verifySignedProfile, canonicalProfileJson } from "@satspath/wasm";
import { buildQrPayload } from "../qr";

export type QuoteResponse =
  | { status: "ok"; recipient: QuoteRecipient; selected_method: PaymentMethod; fee_sats: number; eta: number }
  | { status: "not_registered"; invite: Invite }
  | { status: "no_route"; reason: string }
  | { status: "invalid_signature"; recipient: QuoteRecipient };

export interface QuoteRecipient {
  alias: string;
  verified: boolean;
  profile_signature_verified: boolean;
  identifier_verified: boolean;
  identifier_verification: string;
  fingerprint: string;
}

export interface Invite {
  alias_hash: string;
  amount_sats: number;
  created_at: number;
  expires_at: number;
  claim_url: string;
  warning: string;
  sender_signature?: string;
  sender_pubkey?: string;
}

export async function quote(recipient: string, amount_sats: number): Promise<QuoteResponse> {
  // 1. Resolve via resolver chain
  const resolvers = [
    new LocalRegistryResolver(),
    new Bip353Resolver(),
    new HttpWellKnownResolver(),
    new NostrNip05Resolver(),
  ];

  let signed: SignedPaymentProfile | null = null;
  for (const r of resolvers) {
    try {
      signed = await r.resolve_alias(recipient);
      break;
    } catch { continue; }
  }
  if (!signed) {
    return notRegisteredInvite(recipient, amount_sats);
  }

  // 2. Verify signature (WASM)
```

```

const profileJson = canonicalProfileJson(JSON.stringify(signed.profile));
const verified = await verifySignedProfile(profileJson);
const recipientInfo = buildRecipientInfo(signed.profile, verified);
if (!verified) {
  return { status: "invalid_signature", recipient: recipientInfo };
}

// 3. Expiry check
if (signed.profile.expires_at && Date.now() / 1000 > signed.profile.expires_at) {
  return { status: "no_route", reason: "Profile expired." };
}

// 4. Route selection
const fees = await fetchFeeEstimate();
let route: RouteQuote;
try {
  route = await selectRoute({
    alias: recipient,
    amount_sats,
    signed_profile: signed,
    urgency: "normal"
  }, fees);
} catch (e) {
  return { status: "no_route", reason: e.message };
}

// 5. Build QR payload
const qr = buildQrPayload(route.selected_method, amount_sats);

// 6. (Optional) Fetch real BOLT11 for Lightning
let finalQr = qr;
if (route.selected_method.type === "Lightning" && route.selected_method.lightning_address) {
  try {
    finalQr = await fetchBolt11Invoice(route.selected_method.lightning_address, amount_sats);
  } catch { /* keep pointer */ }
}

return {
  status: "ok",
  recipient: recipientInfo,
  selected_method: route.selected_method,
  fee_sats: route.estimated_fee_sats,
  eta: route.estimated_confirmation,
  reason: route.reason,
  qr: finalQr,
  execution: route.execution,
  wallet_hint: route.wallet_hint
};
}

```

3.5 QR Payload Builder (Port build_qr_payload)

File: packages/router/src/qr.ts

```

export function buildQrPayload(method: PaymentMethod, amount_sats: number): string {
  switch (method.type) {
    case "Lightning":
      return method.lnurl || method.lightning_address || method.bolt12!;
    case "Onchain":
      const addr = method.silent_payment_pubkey || method.address!;
      const btc = (amount_sats / 100_000_000).toFixed(8);
      return `bitcoin:${addr}?amount=${btc}`;
    case "Ark":
      return `ark:${encodeURIComponent(method.pubkey)}?server=${encodeURIComponent(method.server)}&amount=${amount_sats}`;
  }
}

```

3.6 PWA Integration (Arkade Wallet)

File: arkade-wallet/src/services/satspath.ts

```

import { quote, QuoteResponse } from "@satspath/router";
import { verifyVtxoChain, storeExitData } from "@arkade/sdk"; // your existing SDK

export async function getPaymentQuote(alias: string, amountSats: number): Promise<QuoteResponse> {
  return quote(alias, amountSats);
}

export async function executePayment(response: QuoteResponse): Promise<{ txid: string }> {
  const { selected_method, qr, execution } = response;

  switch (selected_method.type) {
    case "Lightning":
      // qr is BOLT11 invoice or LNURL
      if (qr.startsWith("lnbc")) {
        return await ldkSendPayment(qr); // your LDK wrapper
      } else {
        // LNURL → fetch invoice → pay
        const invoice = await fetchLnurlInvoice(qr, response.fee_sats);
        return await ldkSendPayment(invoice);
      }

    case "Onchain":
      // qr is BIP-21 URI
      return await bdkSendPayment(qr); // your BDK/wasm-bitcoin wrapper

    case "Ark":
      // qr is ark: URI → Arkade SDK
      // Arkade SDK does: verify VTXO DAG → store exit data → execute transfer
      const arkResult = await arkadeSendPayment(qr); // your Arkade SDK wrapper
      // Verify received VTXO on receive side
      if (arkResult.vtxo) {
        await verifyVtxoChain(arkResult.vtxo.outpoint, arkadeIndexer, arkadeOnchain);
        await storeExitData(arkResult.vtxo);
      }
      return { txid: arkResult.txid };
  }
}

```



```

    default:
      throw new Error(`Unsupported rail: ${selected_method.type}`);
  }
}

```

3.7 Ark Receive Flow (Arkade SDK)

File: arkade-wallet/src/services/arkReceive.ts

```

import { onReceiveVtxo } from "@arkade/sdk"; // your existing function

export async function handleArkReceive(arkUri: string, amountSats: number) {
  // Parse ark:pubkey?server=...&amount=...
  const { pubkey, server, amount } = parseArkUri(arkUri);

  // Arkade SDK: fetch VTXO chain from indexer, verify, store exit data
  const vtxo = await onReceiveVtxo(
    { txid: "...", vout: 0 }, // outpoint from ASP notification
    arkadeIndexer,
    arkadeOnchain,
    localStorageAdapter // implements StorageProvider
  );

  // UI shows "VTXO verified, sovereign exit ready"
  return vtxo;
}

```

4. File Tree (New Files)

```

satspath/
  sdk/
    wasm/                                # Existing, extend crypto.rs
      pkg/                               # wasm-pack output
    resolvers/                           # NEW: @satspath/resolvers
      package.json
      tsconfig.json
      src/
        types.ts
        index.ts
        localRegistry.ts
        bip353.ts
        httpWellKnown.ts
        nostrNip05.ts
        p2p.ts
      tests/
    router/                              # NEW: @satspath/router
      package.json
      tsconfig.json
      src/
        types.ts
        router.ts
        fees.ts

```

```

    qr.ts
    quote.ts
    index.ts
  tests/
crates/satspath-wasm/src/crypto.rs # ADD exports

arkade-wallet/ (your PWA)
  src/
    services/
      satspath.ts      # NEW: quote + execute
      arkReceive.ts    # NEW: VTXO verification
      ldk.ts / bdk.ts  # Your existing wrappers
    components/
      SendScreen.tsx   # Uses satspath.ts

```

5. Testing Checklist (MVP)

Test	Command
Resolver chain resolves rodrigo@satspath.dev via HTTPS well-known	<code>npm test @satspath/resolvers</code>
WASM verifies valid signature, rejects tampered profile	<code>npm test @satspath/wasm</code>
Router selects Lightning for 21k sats, on-chain for 500k (low fees), Ark for 500k (high fees)	<code>npm test @satspath/router</code>
<code>quote()</code> returns <code>status: "ok"</code> with valid QR for each rail	<code>npm test @satspath/router</code>
<code>quote()</code> returns <code>not_registered</code> with invite for unknown alias	<code>npm test @satspath/router</code>
<code>quote()</code> returns <code>invalid_signature</code> for tampered profile	<code>npm test @satspath/router</code>
PWA Send screen: input alias → shows QR → Lightning pay via LDK works	Manual E2E
PWA Receive Ark: ASP sends VTXO → SDK verifies DAG → exit data stored	Manual E2E (testnet)
<code>satspath-wasm</code> builds for <code>wasm32-unknown-unknown</code> + <code>web</code> target	<code>wasm-pack build --target web</code>

6. Timeline (4–6 Weeks)

Week	Deliverable
1	<code>@satspath/resolvers</code> package: all 4 resolvers + tests
2	Extend <code>satspath-wasm</code> crypto exports + publish <code>@satspath/wasm</code>
3	<code>@satspath/router</code> package: port router + fees + QR + quote + tests
4	PWA integration: <code>satspath.ts</code> service, Send screen wiring
5	Ark receive flow: VTXO verification + sovereign exit storage
6	E2E testing on testnet, bug fixes, docs

7. Risks & Mitigations

Risk	Mitigation
Nostr resolver needs relay connectivity in browser	Use CORS-enabled relays (wss://nostr-pub.wellorder.net, wss://relay.nostr.band) or proxy
BIP-353 DoH blocked by some networks	Fallback to HTTPS well-known; allow user to configure DoH provider
WASM bundle size	<code>wasm-opt -Oz</code> + only export needed functions (~50KB gzipped)
Ark mainnet not ready	Router returns <code>execution: TestnetExperimental</code> ; UI shows “Testnet Preview” badge
LDK/Breez/BDK WASM not integrated yet	MVP uses BIP-21 URI + external wallet fallback (manual wallet handoff)

8. Definition of Done (MVP)

- ☐ `npm run quote -- alice@example.com 21000` returns valid `QuoteResponse` in `<2s`
- ☐ PWA Send screen: paste alias → shows QR → “Pay with Lightning” opens LDK wallet
- ☐ PWA Receive Ark: testnet ASP sends VTXO → SDK verifies → “Sovereign exit ready” badge
- ☐ All resolver/router tests pass in CI
- ☐ `@satspath/wasm` published to npm (or local `file:` dep)
- ☐ No Rust binary required in PWA build pipeline